

realworld_approx

May 27, 2026

1 ANOVA-autoapproximation on real world data

To this point, when doing an ANOVA-approximation of a real world function, it was necessary to use prechosen bandwidths. Now there is a new method provided by the library `pyANOVAapprox` that chooses the optimal bandwidths automatically.

In the following notebook, we will explain the ANOVA-approximation, especially the new method `autoapproximate`, step by step and compare its results to the ones generated with prechosen bandwidths.

Note: To make things easier, we start indexing at 0 instead of 1.

1.0.1 At first, we have to import the necessary libraries.

As we are going to use matrices and vectors, we will need the `numpy`-library. For other mathematical operations, we also use the `math`-library.

```
[1]: import numpy as np
import math
```

For plotting we will need the `matplotlib.pyplot`-library.

```
[2]: import matplotlib.pyplot as plt
```

And of course, we can't forget the most important library, that contains all necessary approximation methods: the `pyANOVAapprox`-library.

```
[3]: import pyANOVAapprox as ANOVAapprox
```

1.0.2 Now, we need the data, we can run our tests on.

In this notebook, we provide five different datasets the user can choose from. All of them are collected from actual real world experiments.

1. **Airfoil-Self-Noise (asn):** This dataset originates from the NASA and contains data about NACA airfoils for different wind tunnel speeds and angles of attack. We aim to find a model that is able to predict the scaled sound pressure level of the self-noise in decibels. The data contains 5 attributes and 1502 nodes.
2. **Ailerons (ail):** The Ailerons dataset describes the control problem of flying a F16 aircraft. The attributes describe the status of the aircraft while we aim to predict the control action

on its ailerons.

We have 40 attributes and 13748 samples.

3. **California Housing (ch):** The dataset describes the prices for houses in California using data about the block groups from 1990 census.

Using 8 attributes and 20639 cases, we aim to predict the median house price for the area.

4. **Energy Efficiency Cooling Load (enc)/ Energy Efficiency Heating Load (enh):** The dataset describes the energy efficiency of houses by 8 attributes and two values to predict, the cooling load and the heating load. Therefore, we have two problems:

ENC with 8 attributes and one continuous value to predict for the cooling load and

ENH with 8 attributes and one continuous value to predict for the heating load.

The dataset contains 767 samples.

You can now choose one of the five datasets, which the approximation will be performed on. Therefore put the abbreviation written in brackets behind each dataset into the following code block.

```
[4]: #choose data (choice between "asn", "ail", "ch", "enc", "enh")
dataset = "ch"
```

Next, we import the chosen data from the corresponding external file.

X is a matrix containing all the data nodes as columns, meaning that $X \cdot e_i$ (e_i here being the i -th standard basis vector) gives the i -th data node as a column vector for $i \leq \# \text{datanodes}$.

y is the vector containing the corresponding value for each node, e.g.

$$Xe_i \mapsto y_i$$

for $i \leq \# \text{datanodes}$.

```
[5]: file = dataset + "_data.npz"

data = np.load(file)

X = data["X"].transpose()
y = data["y"]

data.close()
```

From now on, we will save the number of datanodes in the variable n . So we define

$$n := \# \text{datanodes}.$$

```
[6]: #save number of datanodes
n = y.shape[0]
```

To build a good model, we need some datanodes to train it and some datanodes to test it later. For that purpose we will split our set of datanodes and the set of the corresponding values randomly into two disjunct subsets, one for training and one for testing.

To do that, a permutation $\pi \in S_n$ is chosen randomly. Then the columns of X and the rows of y are swapped according to π , meaning that

$$X_{i,\bullet} \mapsto X_{\pi(i),\bullet}$$

and

$$y_i \mapsto y_{\pi(i)}$$

for $0 \leq i \leq n-1$.

```
[7]: #choose a permutation at random
permutation = np.random.permutation(n)

#permute X and y
X_permutated = X.transpose()[permutation]
y_permutated = y[permutation]
```

Finally, by setting `split_percentage` to some percentage between 0 and 100, the matrices X and y are split at the index $\lceil \frac{\text{split_percentage}}{100} \cdot n \rceil$. This means, that we are going to use `split_percentage` percent of the nodes for training and the rest for testing, that means

$$X_{\text{train}} := \begin{bmatrix} \vdots & & \vdots \\ X_{\pi^{-1}(0),\bullet} & \cdots & X_{\pi^{-1}(\lceil \frac{\text{split_percentage}}{100} \cdot n \rceil),\bullet} \\ \vdots & & \vdots \end{bmatrix},$$

$$X_{\text{test}} := \begin{bmatrix} \vdots & & \vdots \\ X_{\pi^{-1}(\lceil \frac{\text{split_percentage}}{100} \cdot n \rceil + 1),\bullet} & \cdots & X_{\pi^{-1}(n-1),\bullet} \\ \vdots & & \vdots \end{bmatrix}.$$

And for our vector containing the values, we get

$$y_{\text{train}} := \begin{bmatrix} y_{\pi^{-1}(0)} \\ \vdots \\ y_{\pi^{-1}(\lceil \frac{\text{split_percentage}}{100} \cdot n \rceil)} \end{bmatrix}$$

and

$$y_{\text{test}} := \begin{bmatrix} y_{\pi^{-1}(\lceil \frac{\text{split_percentage}}{100} \cdot n \rceil + 1)} \\ \vdots \\ y_{\pi^{-1}(n-1)} \end{bmatrix}.$$

Important: The user can choose this ratio between training points and testing points freely. But if `None` is provided as `split_percentage`, we will split the datanodes according to the paper.

```
[8]: #choose split_percentage
split_percentage = 90

#split according to paper
if split_percentage == None:
    if dataset == "asn":
        split_percentage = 80
```

```

elif dataset == "ail" or dataset == "ch":
    split_percentage = 50
else:
    split_percentage = 70

#setting the index where the data will be split
split_index = int(np.ceil(split_percentage/100*n))

#creating the new arrays
X_train, X_test = X_permutated[:split_index].transpose(), u
    ↪ X_permutated[split_index:].transpose()
y_train, y_test = y_permutated[:split_index], y_permutated[split_index:]

```

1.0.3 The Approximation Procedure

Until now, we have just “prepared” our data. In the following, we will outline the approximation process.

Consider a function

$$f \in L_2(D^d) := \left\{ f : D^d \rightarrow \mathbb{K} : \|f\|_{L_2(D^d)} := \sqrt{\int_{D^d} |f(x)|^2 dx} < \infty \right\}$$

where $d \in \mathbb{N}$ is the so called spatial dimension. We set $D = \mathbb{T}$, which we identify by the interval $[-0.5, 0.5]$, if f is 1-periodic and $D = [0, 1]$ if f is non-periodic. In the non-periodic case we will assume $\mathbb{K} = \mathbb{R}$ while in the periodic case $\mathbb{K} = \mathbb{C}$ is allowed. Furthermore, we use the conventional scalar product

$$\langle f, g \rangle := \int_{D^d} f(x) \overline{g(x)} dx$$

between two functions $f, g \in L_2(D^d)$.

For any given real world data, we assume that behind the data is a function $f \in L_2(D^d)$. Knowing that function can give us crucial information for creating predictions based on former observations. But normally we don’t know this function and/ or it can’t be determined exactly. For that reason we at least try to get an approximation function which we can then use to build a good model according to the data.

We assume that our data follows a function $f \in L_2(D^d)$. We are hence interested in a function $\tilde{f}$ with

$$\tilde{f}(x) \approx f(x).$$

Now, consider a complete orthonormal system $\{\phi_{\mathbf{k}}\}_{\mathbf{k} \in \mathbb{Z}^d}$ with tensor product structure, meaning that we have a complete orthonormal system $\{\eta_k\}_{k \in \mathbb{Z}}$ in $L_2(D)$ and

$$\phi_{\mathbf{k}} = \prod_{i=1}^d \eta_{k_i}(x_i).$$

Since $L_2(D^d)$ together with the scalar product given above forms a Hilbert space, an $f \in L_2(D^d)$ can be written as

$$f(x) = \sum_{\mathbf{k} \in \mathbb{Z}^d} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(x)$$

with $c_{\mathbf{k}}(f) = \langle f, \phi_{\mathbf{k}} \rangle$. Parseval's identity would also yield

$$\|f\|_{L_2(D^d)}^2 = \sum_{\mathbf{k} \in \mathbb{Z}^d} |c_{\mathbf{k}}(f)|^2.$$

If we now set $\mathcal{D} = \{0, 1, \dots, d-1\}$, then we know by the classical ANOVA decomposition that f can be uniquely decomposed as

$$f(x) = \sum_{\mathbf{u} \subseteq \mathcal{D}} f_{\mathbf{u}}(x)$$

where

$$f_{\mathbf{u}} = f_{\mathbf{u}}(x_{\mathbf{u}}) := \sum_{\substack{\mathbf{k} \in \mathbb{Z}^d \\ \text{supp } \mathbf{k} = \mathbf{u}}} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(x).$$

We call $f_{\mathbf{u}}$ an ANOVA term and identify $\text{supp } \mathbf{k} := \{s \in \mathcal{D} : k_s \neq 0\}$.

Note that we have $|\mathcal{P}(\mathcal{D})| = 2^d$ summands in $\sum_{\mathbf{u} \subseteq \mathcal{D}} f_{\mathbf{u}}(x)$. This means, the number of terms grows exponentially in d showing an expression of the curse of dimensionality in the decomposition. To improve efficiency in the approximation we will somehow need to limit the number of summands. So as a workaround we only take subsets of ANOVA terms $U \subseteq \mathcal{P}(\mathcal{D})$ into account.

So how do we get such a $U \subseteq \mathcal{P}(\mathcal{D})$ to start with? At the beginning, we don't know anything about the function and its behaviour. So there are not many possibilities to choose our U . We will proceed by setting a so called superposition threshold $d_s \in \mathcal{D}$ with $d_s < d$. By default, we set

$$d_s = 2$$

but the user can choose another d_s . So in the following cell, you set a d_s . Write **None**, if you don't want to set a special d_s .

```
[9]: #choose ds here
ds = 2

#if no ds is provided we will proceed with ds = 2
if ds == None:
    ds = 2
```

Now, with our d_s set, we define

$$U_{d_s} := \{\mathbf{u} \subseteq \mathcal{D} : |\mathbf{u}| \leq d_s\}$$

and choose this set U_{d_s} as our initial U . This cuts off all the higher-dimensional interactions and leaves us with only the lower-dimensional ones. In the most cases, we get a really good approximation and the error is minimal. By that procedure, we not longer sum over all $\mathbf{u} \subseteq \mathcal{D}$ but only those sets in U_{d_s} . So we assume

$$f(x) \approx \sum_{\mathbf{u} \in U_{d_s}} f_{\mathbf{u}}(x) = \sum_{\mathbf{u} \in U_{d_s}} \sum_{\substack{\mathbf{k} \in \mathbb{Z}^d \\ \text{supp } \mathbf{k} = \mathbf{u}}} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(x)$$

by the definition of the ANOVA terms. You notice quickly, that limiting ourselves to $U_{d_s} \subseteq \mathcal{D}$ has reduced the number of summands of the outer sum, but we still have infinitely many summands for every $\mathbf{u} \in U_{d_s} \setminus \emptyset$ in the inner sum.

As a consequence, we have to limit ourselves to finitely many terms in the inner sum. We do that by choosing only a finite subset

$$P_{\mathbf{u}} I_{\mathbf{u}} \subseteq \{\mathbf{k} \in \mathbb{Z}^d : \text{supp } \mathbf{k} = \mathbf{u}\}$$

for every $\mathbf{u} \in U_{d_s}$. For that choose $I_{\mathbf{u}} \subseteq (\mathbb{Z} \setminus \{0\})^{|\mathbf{u}|}$. Then

$$P_{\mathbf{u}} I_{\mathbf{u}} := \{\mathbf{k} \in \mathbb{Z}^d : \mathbf{k}_{\mathbf{u}} \in I_{\mathbf{u}}, \mathbf{k}_{\mathbf{u}^c} = \mathbf{0}\}$$

is a set that fulfills the conditions.

So now we have

$$f(x) \approx \sum_{\mathbf{u} \in U_{d_s}} \sum_{\mathbf{k} \in P_{\mathbf{u}} I_{\mathbf{u}}} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(x).$$

As both sums are finite we can also write

$$f(x) \approx \sum_{\mathbf{k} \in I(U)} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(x)$$

where $I(U)$ is the union

$$I(U) = \bigcup_{\mathbf{u} \in U_{d_s}} I_{\mathbf{u}}.$$

So how do we choose those sets $I_{\mathbf{u}}$? We do that by taking an even parameter $N_{\mathbf{u}} \in 2\mathbb{N}$ and setting

$$I_{\mathbf{u}}^{\text{per}} = \left\{ -\frac{N_{\mathbf{u}}}{2}, \dots, -1, 1, \dots, \frac{N_{\mathbf{u}}}{2} - 1 \right\}$$

if f is periodic and

$$I_{\mathbf{u}}^{\text{non-per}} = \{1, 2, \dots, N_{\mathbf{u}} - 1\}$$

if f is non-periodic. But now we run into the problem of selecting a suitable $N_{\mathbf{u}}$.

In theory, you could choose that $N_{\mathbf{u}}$ for every $\mathbf{u} \in U_{d_s}$ separately, but we will set one $N_{\mathbf{u}}$ for all $\mathbf{u} \in U_{d_s}$ with $|\mathbf{u}| = 1$, one $N_{\mathbf{u}}$ for all $\mathbf{u} \in U_{d_s}$ with $|\mathbf{u}| = 2$ and so on. We will call those $N_{\mathbf{u}}$ the bandwidth.

In the following cell you can set those bandwidths by hand. Therefore write a list with length d_s which you put the bandwidths in, e.g. the at the i -th Index, you put the bandwidth $N_{\mathbf{u}}$ for all $\mathbf{u} \in U_{d_s}$ with $|\mathbf{u}| = i + 1$. Keep in mind, that those bandwidths have to be even numbers greater than zero. If you don't want to set any bandwidths by hand, leave the list empty. We will then choose the bandwidths, according to

$$N_{\mathbf{u}} = 2 \cdot \left\lceil \frac{b^{\frac{1}{|\mathbf{u}|}}}{2} \right\rceil$$

where

$$b := \frac{M}{\log_{10}(M) \cdot n_{\text{num}}},$$

where M denotes the number of rows of y_{train} , e.g. $y_{\text{train}} \in \mathbb{R}^M$ and

$$n_{\text{num}} := \sum_{k=1}^{d_s+1} \binom{6}{k}.$$

```
[10]: #put your desired bandwidths into the following list, e.g. bw = [ 6, 4, 2 ] for
      ↪ ds = 2
      #if you don't want to set specific bandwidths yourself, leave the list empty

      bw = []

      #calculating initial bandwidth, if no bandwidths are given
      M = len(y_train)
      if bw == []:
          num = np.sum([math.comb(6, k) for k in np.arange(1, ds + 1)])
          b = M / (math.log10(M) * num)
          for a in range(ds):
              bw.append(math.floor(np.power(b, 1/(a + 1))/2) * 2)
```

As we now have a finite sum, we can finally look for a way to determine the coefficients in

$$f(x) \approx \sum_{\mathbf{k} \in I(U)} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(x).$$

From the assumption, that our f should approximate our data, we get that

$$(y_{\text{train}})_i = \sum_{\mathbf{k} \in I(U)} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(X_{\bullet, i}).$$

This motivates to write the equations rowwise underneath each other to get

$$\begin{aligned} (y_{\text{train}})_0 &= \sum_{\mathbf{k} \in I(U)} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(X_{\bullet, 0}) \\ &\vdots \\ (y_{\text{train}})_{M-1} &= \sum_{\mathbf{k} \in I(U)} c_{\mathbf{k}}(f) \cdot \phi_{\mathbf{k}}(X_{\bullet, M-1}). \end{aligned}$$

As the coefficients are the same in every equation, we can write this system of equations in the classic matrix form:

$$\mathbf{y} := y_{\text{train}} = \begin{bmatrix} \phi_{\emptyset}(X_{\bullet, 1}) & \cdots & \phi_{\mathbf{k}}(X_{\bullet, 1}) & \cdots \\ \vdots & & \vdots & \\ \phi_{\emptyset}(X_{\bullet, M}) & \cdots & \phi_{\mathbf{k}}(X_{\bullet, M}) & \cdots \end{bmatrix} \cdot \begin{bmatrix} c_{\emptyset} \\ \vdots \\ c_{\mathbf{k}} \\ \vdots \end{bmatrix}.$$

By defining

$$\mathbf{F}(X, I(u)) := (\phi_{\mathbf{k}}(X_{\bullet, i}))_{\mathbf{k} \in I(U), i \in \{1, \dots, M\}}$$

we can rewrite our linear equation as

$$0 \stackrel{!}{=} \mathbf{y} - \mathbf{F}(X, I(U))\hat{\mathbf{f}}$$

where $\hat{\mathbf{f}}$ holds our coefficients that are to be determined. Now in the most cases, we can't solve that system directly, as $\mathbf{F}(X, I(u))$ won't be a square matrix and even then we don't know if it is invertible. So instead we determine the coefficients by solving the least-squares problem:

$$\hat{\mathbf{f}} = (\hat{f}_{\mathbf{k}})_{\mathbf{k} \in I(U)} = \arg \min_{\hat{\mathbf{g}} \in \mathbb{K}^{|I(U)|}} \|\mathbf{y} - \mathbf{F}(X, I(U))\hat{\mathbf{g}}\|_2^2.$$

If $\mathbf{F}(X, I(u))$ has full rank, the problem at least has a unique solution and $\hat{f}_{\mathbf{k}} \approx c_{\mathbf{k}}(f)$. If the oversampling is large enough, meaning that $M > |I(U)|$, e.g. we have much more columns than rows, we will assume full rank for $\mathbf{F}(X, I(u))$. For an arbitrary chosen $x \in D^d$, we then get our approximation through

$$f(x) \approx S(X, I(U))f(x) = \sum_{\mathbf{k} \in I(U)} \hat{f}_{\mathbf{k}} \cdot \phi_{\mathbf{k}}(x).$$

To solve the least-squares problem efficiently, we are going to use the iterative LSQR solver. As the LSQR solver needs to multiply with $\mathbf{F}(X, I(u))$, its adjoint $\mathbf{F}^*(X, I(u))$ in the periodic case and its transposed $(\mathbf{F}(X, I(u)))^T$ in the non-periodic case, we provide it with a method for fast matrix multiplication. This method is based on the idea of Grouped Transformation which itself is based on the NFFT or the NFCT. As the grouped transformations provide oracle functions for the multiplications, we can even use the matrix-free variant of LSQR and don't need to construct $\mathbf{F}(X, I(u))$ explicitly.

Now it is possible for $\mathbf{F}(X, I(u))$ to not have full rank, especially if $|I(U)| > M$, e.g. we have more rows than columns. In this case we can add l_2 or *Tikhonov* regularization. We then solve the modified least-squares problem:

$$\arg \min_{\hat{\mathbf{g}} \in \mathbb{K}^{|I(U)|}} [\|\mathbf{y} - \mathbf{F}(X, I(u))\hat{\mathbf{g}}\|_2^2 + \lambda \|\hat{\mathbf{g}}\|_2^2],$$

by introducing a regularization parameter $\lambda > 0$. Notice that by setting $\lambda = 0$ we would get our original problem back. Furthermore we get

$$\begin{aligned} \|\mathbf{y} - \mathbf{F}(X, I(u))\hat{\mathbf{g}}\|_2^2 + \lambda \|\hat{\mathbf{g}}\|_2^2 &= \langle \mathbf{y} - \mathbf{F}(X, I(u))\hat{\mathbf{g}}, \mathbf{y} - \mathbf{F}(X, I(u))\hat{\mathbf{g}} \rangle + \langle \sqrt{\lambda}\hat{\mathbf{g}}, \sqrt{\lambda}\hat{\mathbf{g}} \rangle \\ &= \left\langle \begin{bmatrix} \mathbf{y} - \mathbf{F}(X, I(u))\hat{\mathbf{g}} \\ \sqrt{\lambda}\hat{\mathbf{g}} \end{bmatrix}, \begin{bmatrix} \mathbf{y} - \mathbf{F}(X, I(u))\hat{\mathbf{g}} \\ \sqrt{\lambda}\hat{\mathbf{g}} \end{bmatrix} \right\rangle \\ &= \left\| \begin{pmatrix} \mathbf{y} \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{F}(X, I(u)) \\ \sqrt{\lambda}\mathbf{I} \end{pmatrix} \hat{\mathbf{g}} \right\|_2^2, \end{aligned}$$

where $\mathbf{0}$ is the zero vector $\in \mathbb{R}^{|I(U)|}$ and $\mathbf{I}$ is the identity matrix $\in \mathbb{R}^{|I(U)| \times |I(U)|}$. So as a consequence, we can use the LSQR solver for the modified problem too. As

$$\begin{pmatrix} \mathbf{F}(X, I(u)) \\ \sqrt{\lambda}\mathbf{I} \end{pmatrix} \in \mathbb{K}^{(M+|I(U)|) \times |I(U)|}$$

the matrix has more rows than columns. It follows that, if this matrix has full column rank, we again get a unique solution from solving the least-squares problem.

In the following code cell, you can type in a list of finitely many λ for which the least-squares problem should be solved. As we test our model afterwards we will automatically choose the λ , which gives the model with the least error. If you don't want to set any λ leave the list empty. The least-squares problem will only be solved for $\lambda = 0$, e.g. the case without regularization.

```
[11]: #choose the lambdas for regularization, e.g. lambdas = [ 0.5, 1.0, 2.0 ]
      lambdas = [ 0.0 ]

      #setting the list, if no lambda is given
      if lambdas == []:
          lambdas = [ 0.0 ]
```

Now before we can solve the least-squares problem, we need to choose our basis, e.g. the orthonormal system $\{\phi_{\mathbf{k}}\}_{\mathbf{k} \in \mathbb{Z}^d}$. For that you have to select one of the following bases:

1. **The Fourier System (per):**

$$\phi_{\mathbf{k}}^{\text{exp}}(x) = e^{2\pi i \mathbf{k} \cdot x}$$

for periodic functions,

2. **The cosine system (cos):**

$$\phi_{\mathbf{k}}^{\text{exp}}(x) = \sqrt{2^{|\text{supp } \mathbf{k}|}} \prod_{s \in \text{supp } \mathbf{k}} \cos(\pi k_s x_s)$$

for non-periodic functions,

3. **The Chebyshev system (cheb):**

$$\phi_{\mathbf{k}}^{\text{exp}}(x) = \sqrt{2^{|\text{supp } \mathbf{k}|}} \prod_{s \in \text{supp } \mathbf{k}} \cos(k_s \arccos(2x_s - 1))$$

for non-periodic functions.

In the following code cell, you can determine, which bases you want to use by writing the abbreviation given above in quotation marks, e.g. `basis = "cos"` for the cosine system. If you don't want to choose a basis, write `None`. Then the cosine basis is used.

In the case of the Fourier basis, the nodes need to be between -0.5 and 0.5 . In the case of the cosine or Chebyshev basis, the nodes need to be between 0 and 1 . Depending on the choice of the basis the nodes are automatically scaled.

```
[12]: #write the abbreviation of the basis you want to use in quotation marks, e.g.
      ↪as a string of letters
      basis = None
```

```

#choose basis if None is given
if basis == None:
    basis = "cos"

#scaling of the nodes
max = np.max([np.max(X_train), np.max(X_test)])
min = np.min([np.min(X_train), np.min(X_test)])
if min < 0 or max > 1:
    X_train = (X_train - min) / (max - min)
    X_test = (X_test - min) / (max - min)
if basis == "per":
    X_train /= 4
    X_test /= 4

```

Now we can initiate the process, meaning that we compute the coefficients $c_{\mathbf{k}}(f)$ by taking the training the training data as the input.

```

[13]: #approximation
ads = ANOVAapprox.approx(np.transpose(X_train), y_train, lam = lambdas, ds = _
    _ds, N = bw, basis = basis)
ANOVAapprox.approx.approximate(ads)

```

Trough this approximation we have gained crucial information about our f . We want to use that information to improve our U and run the whole approximation again.

So at first we want to somehow “measure the influence/ importance” of each $\mathbf{u} \in U$. Therefore, we define the variance of f as

$$\sigma^2(f) := \|f\|_{L_2(D^d)}^2 - |c_0(f)|^2 = \sum_{\mathbf{k} \in \mathbb{Z}^d \setminus \{0\}} |c_{\mathbf{k}}(f)|^2.$$

Notice that for every $u \in U$ the ANOVA terms $f_{\mathbf{u}}$ are functions $\in L_2(D^d)$ as well so we can take their variance as

$$\sigma^2(f_{\mathbf{u}}) = \|f_{\mathbf{u}}\|_{L_2(D^d)}^2.$$

The constant term $|c_0(f_{\mathbf{u}})|$ vanishes as it is equal to zero by definition.

The relative importance of each $\mathbf{u} \in U$ is now measured through global sensitivity indices (GSIs), defined as

$$\rho(\mathbf{u}, f) := \frac{\sigma^2(f_{\mathbf{u}})}{\sigma^2(f)}.$$

By definition we have $\rho(\mathbf{u}, f) \in [0, 1]$. As we have only approximated the coefficients, we will use the GSIs given by the approximation, e.g. $\rho(\mathbf{u}, S(X, I(U))f)$.

Now we are also interested in how much influence one variable $i \in \mathcal{D}$ has to the variance of the

function. We achieve this by considering all terms $\mathbf{u} \in U$ with $i \in \mathbf{u}$ and summing their GSIs:

$$\sum_{\mathbf{u} \in \{\mathbf{v} \in U : i \in \mathbf{v}\}} \rho(\mathbf{u}, S(X, I(U))f).$$

It can happen that a variable seems important if it occurs in many rather unimportant terms of the same cardinality in contrast to other variables with some terms of that order not appearing in U . Therefore, we divide each summand by the amount of subsets of U_{d_s} with i in it and the same cardinality as the current $\mathbf{u}$:

$$\sum_{\mathbf{u} \in \{\mathbf{v} \in U : i \in \mathbf{v}\}} \frac{\rho(\mathbf{u}, S(X, I(U))f)}{|\{\mathbf{v} \in U : |\mathbf{u}| = |\mathbf{v}|, i \in \mathbf{v}\}|}.$$

We could already take that as our attribute ranking, but we choose to divide by a certain normalization to achieve $\sum_{i \in \mathcal{D}} r(i) = 1$, where

$$r(i) = \frac{\sum_{\mathbf{u} \in \{\mathbf{v} \in U : i \in \mathbf{v}\}} \frac{\rho(\mathbf{u}, S(X, I(U))f)}{|\{\mathbf{v} \in U : |\mathbf{u}| = |\mathbf{v}|, i \in \mathbf{v}\}|}}{\sum_{\mathbf{u} \in U} \left(\sum_{i \in \mathbf{u}} \frac{1}{|\{\mathbf{v} \in U : |\mathbf{u}| = |\mathbf{v}|, i \in \mathbf{v}\}|} \right) \rho(\mathbf{u}, S(X, I(U))f)}.$$

In the following two diagrams, we will plot the attribute ranking $r(i)$ for each $i \in \mathcal{D}$ and then the GSIs for every $\mathbf{u} \in U_{d_s}$. For better comparison, we plot the attribute ranking and the GSIs for every regularization parameter λ into the same diagram, if there are five or fewer λ given. If more are given, we will plot the attribute ranking and gsis for each λ in a different diagram.

```
[14]: ri = ads.get_AttributeRanking()
      gsis = ads.get_GSI()

      if len(lambdas) <= 5:
          colors = [ "orange", "green", "red", "blue", "purple" ]

          #creating raw plot
          fig, (ax1, ax2) = plt.subplots(2,1)

          #plotting attribute ranking
          #array for xaxis
          xaxis = np.arange(1, X.shape[0] + 1, 1)

          #plotting the attribute ranking
          for l in range(len(lambdas)):
              ax1.scatter(xaxis + 0.15*l, ri[lambdas[l]], c = colors[l], marker = '^',
              label = "$\\lambda = $" + str(lambdas[l]))
              for i in range(len(xaxis)):
                  ax1.plot([i + 1 + 0.15*l, i + 1 + 0.15*l], [0, ri[lambdas[l]][i]], c = colors[l])
```

```

ax1.set_xticks(xaxis + (len(lambdas) - 1)*0.075)
ax1.set_xticklabels(xaxis - 1)
ax1.set_ylabel("$r(i)$")
ax1.set_title("attribute ranking")
ax1.set_xlim(0.75, len(ri[lambdas[0]]) + 1.5)
ax1.set_ylim(0, 1)
ax1.legend()
ax1.grid()

#plotting gsis
#array for xaxis
xaxis = np.arange(0, len(gsis[lambdas[0]]),1)

#plotting the gsis
for l in range(len(lambdas)):
    ax2.scatter(xaxis + 0.15*l, gsis[lambdas[l]], c = colors[l], marker = '^',
    label = "$\\lambda = $" + str(lambdas[l]))
    for i in range(len(xaxis)):
        ax2.plot([i + 0.15*l, i + 0.15*l], [0, gsis[lambdas[l]][i]], c = colors[l])

ax2.set_xticks(xaxis + (len(lambdas) - 1)*0.075)
ax2.set_xticklabels([tuple(x for x in tup) for tup in ads.get_ActiveSet(np.
zeros((ds)), lam = lambdas[0])[1:])]
ax2.set_ylabel("$\\rho(\\mathbf{u}, S(X, I(U)) f)$")
ax2.set_title("global sensitivity indices")
ax2.set_xlim(-0.5, len(gsis[lambdas[0]]) + 1.5)
ax2.legend()
ax2.grid()

fig.set_figwidth(xaxis.shape[0]/1.2)
fig.set_figheight(10)
fig.subplots_adjust(hspace=0.4)
plt.show()
else:
    for l in range(len(lambdas)):
        color = colors[l % 5]
        #creating raw plot
        fig, (ax1, ax2) = plt.subplots(2,1)

        #plotting attribute ranking
        #array for xaxis
        xaxis = np.arange(1, X.shape[0] + 1, 1)

```

```

#plotting attribute ranking
ax1.scatter(xaxis, ri[lambdas[1]], c = color, marker = "^", label =
↳"$\\lambda = $" + str(lambdas[1]))
for i in xaxis:
    ax1.plot([i,i],[0,ri[lambdas[1]][i - 1]], c = color)
ax1.set_xticks(xaxis - 1)
ax1.set_ylabel("$r(i)$")
ax1.set_title("attribute ranking  $( \\lambda = $" + str(lambdas[1]) +
↳"$)$")
ax1.legend()
ax1.grid()

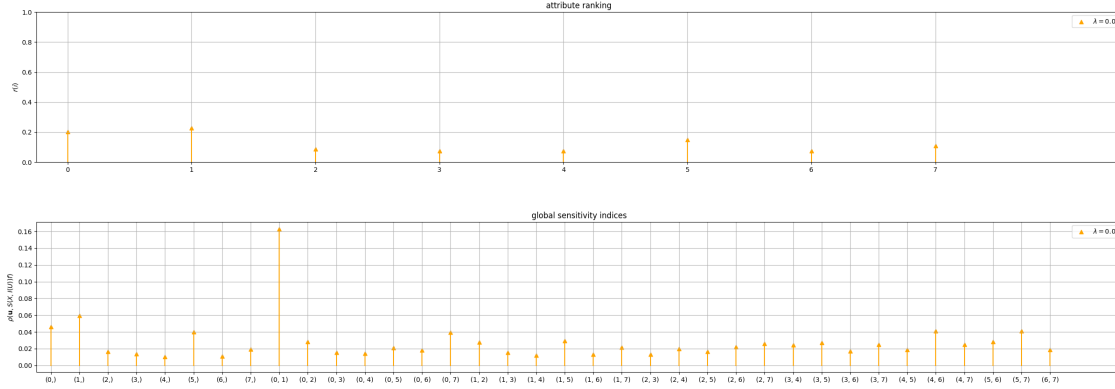
#plotting gsis
#array for xaxis
xaxis = np.arange(0, len(gsis[lambdas[0]]),1)

#plotting gsis
ax2.scatter(xaxis, gsis[lambdas[1]], c = color, marker = "^", label =
↳"$\\lambda = $" + str(lambdas[1]))
for i in xaxis:
    ax2.plot([i,i],[0,gsis[lambdas[1]][i]], c = color)

ax2.set_xticks(xaxis)
ax2.set_xticklabels([tuple(x for x in tup) for tup in ads.
↳get_ActiveSet(np.zeros((ds)), lam = lambdas[0])[1:]])
ax2.set_ylabel("$ \\rho(\\mathbf{u}, S(X, I(U)) f)$")
ax2.set_title("global sensitivity indices  $( \\lambda = $" +
↳str(lambdas[1]) + "$)$")
ax2.legend()
ax2.grid()

fig.set_figwidth(xaxis.shape[0]/1.2)
fig.subplots_adjust(hspace=0.4)
plt.show()

```



At this point, we want to get rid of unimportant terms, meaning subsets $\mathbf{u} \in U_{d_s}$ that have little to no influence on the variance of the function.

So at first, we focus on the attribute ranking. There is the option to remove some $i \in \mathcal{D}$ entirely, if they do not contribute much to the function. For that, you can either write the $i \in \mathcal{D}$ you want to remove down below or set an $\varepsilon > 0$ that cuts off all indices $i \in \mathcal{D}$ with $r(i) < \varepsilon$. The approximation is repeated afterwards with the chosen dimensions removed, e.g. if you remove $i \in \mathcal{D}$, then the data nodes will be reduced to

$$x = (x_0, \dots, x_{i-1}, x_{i+1}, \dots, x_{|\mathcal{D}|-1}) \in \mathcal{D}^{d-1}.$$

The d_s , prechosen bandwidths, lambdas and basis will be kept.

So in the following code cell write down the indices $i \in \mathcal{D}$ you want to remove as a list, e.g. `dr = [ 0, 2]` or, if you want to choose an ε for cutoff, set `dr = epsilon`, e.g. `dr = 0.1`. If you don't want to remove any indices, set `dr = None`. We will then proceed by keeping all indices.

```
[15]: #cutoff indices
dr = None

if dr != None:
    dimensions = len(ri[0])
    if type(dr) == float or type(dr) == int:
        epsilon = dr
        dr = []
        for i in range(dimensions):
            if np.min([ri[l][i] for l in lambdas]) < epsilon:
                dr.append(i)

    #plotting attribute ranking again to visualize cutoff
    if len(lambdas) <= 5:
        colors = [ "orange", "green", "red", "blue", "purple" ]

    #creating raw plot
    fig, ax1 = plt.subplots(1,1)
```

```

    #plotting attribute ranking
    #array for xaxis
    xaxis = np.arange(1, X.shape[0] + 1, 1)

    #plotting the attribute ranking
    for l in range(len(lambdas)):
        ax1.scatter(xaxis + 0.15*l, ri[lambdas[l]], c = colors[l],
↪marker = "^", label = "$\\lambda = $" + str(lambdas[l]))
        for i in range(len(xaxis)):
            ax1.plot([i + 1 + 0.15*l, i + 1 + 0.
↪15*l], [0, ri[lambdas[l]][i]], c = colors[l])
            ax1.plot([0.75, len(ri[lambdas[0]]) + 1.5], [epsilon, epsilon],
↪color = "black", linestyle = "--", label = "$\\varepsilon$")
            ax1.set_xticks(xaxis + (len(lambdas) - 1)*0.075)
            ax1.set_xticklabels(xaxis - 1)
            ax1.set_ylabel("$r(i)$")
            ax1.set_title("attribute ranking")
            ax1.set_xlim(0.75, len(ri[lambdas[0]]) + 1.5)
            ax1.set_ylim(0, 1)
            ax1.legend()
            ax1.grid()

    fig.set_figwidth(10)
    fig.set_figheight(4)
    plt.show()
else:
    for l in range(len(lambdas)):
        color = colors[l % 5]
        #creating raw plot
        fig, ax1 = plt.subplots(1,1)

        #plotting attribute ranking
        #array for xaxis
        xaxis = np.arange(1, X.shape[0] + 1, 1)

        #plotting attribute ranking
        ax1.scatter(xaxis, ri[lambdas[l]], c = color, marker = "^",
↪label = "$\\lambda = $" + str(lambdas[l]))
        for i in range(xaxis):
            ax1.plot([i,i], [0, ri[lambdas[l]][i - 1]], c = color)
            ax1.plot([0.75, len(ri[lambdas[0]]) + 1.5], [epsilon, epsilon],
↪color = "black", linestyle = "--", label = "$\\varepsilon$")
            ax1.set_xticks(xaxis - 1)
            ax1.set_ylabel("$r(i)$")
            ax1.set_title("attribute ranking  $( \\lambda = $" +
↪str(lambdas[l]) + "$)$")

```

```

        ax1.legend()
        ax1.grid()

        fig.set_figwidth(10)
        fig.set_figheight(4)
        plt.show()

#printing indices which have been cut off
    if len(dr) == 0:
        print("For all indices in D we have epsilon < r(i), so no indices have_
        ↪been removed.")
    elif len(dr) == 1:
        print("Approximation on " + str(dimensions - len(dr)) + " dimensions_
        ↪instead of " + str(dimensions) + ". The index " + str(dr) + " has been_
        ↪removed.")
    else:
        print("Approximation on " + str(dimensions - len(dr)) + " dimensions_
        ↪instead of " + str(dimensions) + ". The indices " + str(dr) + " have been_
        ↪removed.")

```

If indices are to be removed, we will remove them from our data nodes and repeat the approximation on the “reduced” nodes. If no indices should be removed, we will skip this step.

```

[16]: if dr != None and dr != []:
        #deleting the indices
        X_train = np.delete(X_train, [i - 1 for i in dr], axis = 0)
        X_test = np.delete(X_test, [i - 1 for i in dr], axis = 0)

        #repeating the approximation on the "reduced" data nodes
        ads = ANOVAapprox.approx(np.transpose(X_train), y_train, lam = lambdas, ds_
        ↪= ds, N = bw, basis = basis)
        ANOVAapprox.approx.approximate(ads)

```

Now, we have reduced the dimensionality of the problem. In the following graph, you can see the gsis again, in order to set an $\varepsilon \in (0, 1)^{d_s}$ for them too. As a consequence we will cutoff all $\mathbf{u} \in U_{d_s}$ with

$$\rho(\mathbf{u}, S(X, I(U))f) \leq \varepsilon_{|\mathbf{u}|}.$$

($\varepsilon_{|\mathbf{u}|}$ denotes the $|\mathbf{u}|$ entry of the vector ε .) We will proceed by setting

$$U(\varepsilon) := \left\{ \mathbf{u} \in U_{d_s} : \rho(\mathbf{u}, S(X, I(U))f) > \varepsilon_{|\mathbf{u}|} \right\}.$$

```

[17]: gsis = ads.get_GSI()

    if len(lambdas) <= 5:
        colors = [ "orange", "green", "red", "blue", "purple" ]

```



```

#creating raw plot
fig, ax2 = plt.subplots(1,1)

#plotting gsis
#array for xaxis
xaxis = np.arange(0, len(gsis[lambdas[0]]),1)

#plotting the gsis
for l in range(len(lambdas)):
    ax2.scatter(xaxis + 0.15*l, gsis[lambdas[l]], c = colors[l], marker = "
↪^", label = "$\\lambda = $" + str(lambdas[l]))
    for i in range(len(xaxis)):
        ax2.plot([i + 0.15*l, i + 0.15*l], [0, gsis[lambdas[l]][i]], c =
↪colors[l])

    ax2.set_xticks(xaxis + (len(lambdas) - 1)*0.075)
    ax2.set_xticklabels([tuple(x for x in tup) for tup in ads.getActiveSet(np.
↪zeros((ds)), lam = lambdas[0])[1:])]
    ax2.set_ylabel("$\\rho(\\mathbf{u}, S(X, I(U)) f)$")
    ax2.set_title("global sensitivity indices")
    ax2.set_xlim(-0.5, len(gsis[lambdas[0]]) + 1.5)
    ax2.legend()
    ax2.grid()

fig.set_figwidth(xaxis.shape[0]/1.2)
fig.set_figheight(5)
plt.show()
else:
    for l in range(len(lambdas)):
        color = colors[l % 5]
        #creating raw plot
        fig, ax2 = plt.subplots(1,1)

        #plotting gsis
        #array for xaxis
        xaxis = np.arange(0, len(ads.getActiveSet(np.zeros((ds)), lam =
↪lambdas[0])) - 1,1)

        #plotting gsis
        ax2.scatter(xaxis, gsis[lambdas[l]], c = color, marker = "^", label =
↪"$\\lambda = $" + str(lambdas[l]))
        for i in xaxis:
            ax2.plot([i,i], [0, gsis[lambdas[l]][i]], c = color)

        ax2.set_xticks(xaxis)
        ax2.set_xticklabels([tuple(x for x in tup) for tup in ads.
↪getActiveSet(np.zeros((ds)), lam = lambdas[0])[1:])]

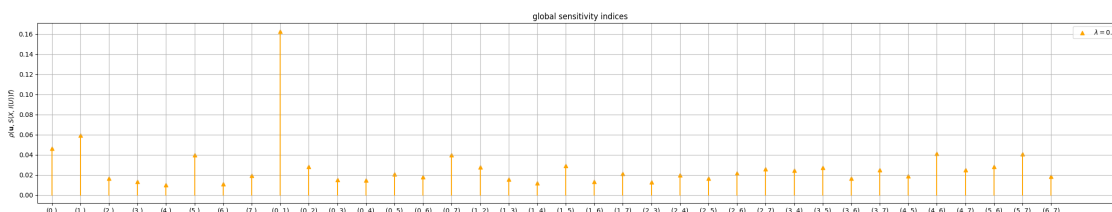
```

```

ax2.set_ylabel("$ \rho(\mathbf{u}, S(X, I(U)) f)$")
ax2.set_title("global sensitivity indices $(\lambda = \text{ } + \text{ } \text{ }$
↪str(lambdas[1]) + "$)$")
ax2.legend()

fig.set_figwidth(xaxis.shape[0]/1.2)
plt.show()

```



In the following code cell, you can choose an epsilon for cut off, e.g. `epsilon = 0.1`. If you don't want to choose an epsilon, we will proceed by taking the mean of all gsis:

$$\varepsilon = \left(\frac{1}{|U_{d_s}|} \sum_{\mathbf{u} \in U_{d_s}} \rho(\mathbf{u}, S(X, I(U))f) \right) \cdot \mathbf{1},$$

where $\mathbf{1}$ is the all-one vector of dimension d_s .

```

[18]: #choosing an epsilon for cut off
epsilon = 0.04

if epsilon == None:
    epsilon = np.min([np.mean(ads.get_GSI(lam = 1)) for l in lambdas])
    lambdamin = np.argmin([np.mean(ads.get_GSI(lam = 1)) for l in lambdas])

#cutting off all dimensions with GSI less than epsilon
U = ads.get_ActiveSet(np.full((ds),epsilon), lam = lambdas[lambdamin])

#printing gsis with epsilon
if len(lambdas) <= 5:
    colors = [ "orange", "green", "red", "blue", "purple" ]

    #creating raw plot
    fig, ax2 = plt.subplots(1,1)

    #plotting gsis
    #array for xaxis

```

```

xaxis = np.arange(0, len(gsis[lambdas[0]]),1)

#plotting the gsis
for l in range(len(lambdas)):
    ax2.scatter(xaxis + 0.15*l, gsis[lambdas[l]], c = colors[l], marker = "
↪^", label = "$\\lambda = $" + str(lambdas[l]))
    for i in range(len(xaxis)):
        ax2.plot([i + 0.15*l,i + 0.15*l],[0,gsis[lambdas[l]][i]], c =
↪colors[l])
    ax2.plot([-0.5, len(gsis[lambdas[0]]) + 1.5], [epsilon, epsilon], color =
↪"black", linestyle = "--", label = "$\\varepsilon$")
    ax2.set_xticks(xaxis + (len(lambdas) - 1)*0.075)
    ax2.set_xticklabels([tuple(x for x in tup) for tup in ads.getActiveSet(np.
↪zeros((ds)), lam = lambdas[0])[1:])]
    ax2.set_ylabel("$ \\rho(\\mathbf{u}, S(X, I(U)) f)$")
    ax2.set_title("global sensitivity indices")
    ax2.set_xlim(-0.5, len(gsis[lambdas[0]]) + 1.5)
    ax2.legend()
    ax2.grid()

fig.set_figwidth(xaxis.shape[0]/1.2)
fig.set_figheight(5)
plt.show()
else:
    for l in range(len(lambdas)):
        color = colors[l % 5]
        #creating raw plot
        fig, ax2 = plt.subplots(1,1)

        #plotting gsis
        #array for xaxis
        xaxis = np.arange(0, len(ads.getActiveSet(np.zeros((ds)), lam =
↪lambdas[0])) - 1,1)

        #plotting gsis
        ax2.scatter(xaxis, gsis[lambdas[l]], c = color, marker = "^", label =
↪"$\\lambda = $" + str(lambdas[l]))
        for i in xaxis:
            ax2.plot([i,i],[0,gsis[lambdas[l]][i]], c = color)

        ax2.set_xticks(xaxis)
        ax2.set_xticklabels([tuple(x for x in tup) for tup in ads.
↪getActiveSet(np.zeros((ds)), lam = lambdas[0])[1:])]
        ax2.set_ylabel("$ \\rho(\\mathbf{u}, S(X, I(U)) f)$")
        ax2.set_title("global sensitivity indices $(\\lambda = $" +
↪str(lambdas[l]) + "$)$")

```

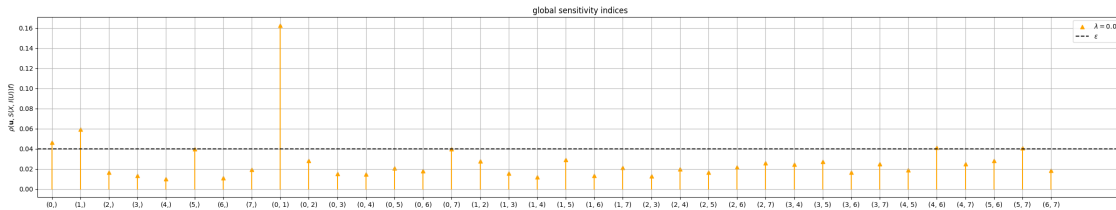
```

ax2.legend()

fig.set_figwidth(xaxis.shape[0]/1.2)
plt.show()

print("Our new U is " + str(U) + ".")

```



Our new U is [(), (0,), (1,), (0, 1), (4, 6), (5, 7)].

With the information gained through the attribute ranking and the gsis we have determined a new U . We now run the new approximation with automatic bandwidth detection in comparison to the previous approximation again. For that, we use the training data, all of the lambdas, the same basis as before and our newly found U . We will also use the same bandwidths as before for the “old” approximation method.

For the autoapproximation of bandwidths the user can set a maximum number of iterations, the approximation will go through. Therefore place a number of iterations in the following code cell, e.g. `maxiter = 3`. If you don't wish to set a maximum number of iterations, we will use five iterations. So place `maxiter = None` instead of a number.

```

[19]: #choose maximum number of iterations for autoapproximation
maxiter = 2

if maxiter == None:
    maxiter = 5

```

Now, we run both methods.

```

[20]: #approximation without automatic bandwidth detection
ads2 = ANOVAapprox.approx(np.transpose(X_train), y_train, lam = lambdas, U = U,
    ↪ N = bw, basis = basis)
ANOVAapprox.approx.approximate(ads2)

#approximation with automatic bandwidth detection
ads_auto = ANOVAapprox.approx(np.transpose(X_train), y_train, lam = lambdas, U =
    ↪ U, basis = basis)
ANOVAapprox.approx.autoapproximate(ads_auto, maxiter = maxiter, verbosity = 10)

```

```

B = 2388.185799993649
===== Iteration 1 =====
bw in () : []
bw in (0,) : [18]
bw in (1,) : [18]
bw in (0, 1) : [6, 6]
bw in (4, 6) : [6, 6]
bw in (5, 7) : [6, 6]

estimated rates for () : D = [] and t = []
estimated rates for (0,) : D = [nan] and t = [nan]
estimated rates for (1,) : D = [nan] and t = [nan]
estimated rates for (0, 1) : D = [nan, nan] and t = [nan, nan]
estimated rates for (4, 6) : D = [nan, nan] and t = [nan, nan]
estimated rates for (5, 7) : D = [nan, nan] and t = [nan, nan]

```

1.0.4 Comparing the results

In the end, we want to compare the results of both approximations. Therefore, we calculate the mean squared error of our model and the test data for both models and all our λ . We then proceed by choosing the model generated by the λ that yields the best result, i.e. the minimal mean squared error:

$$\text{MSE}(y_{\text{test}}, \tilde{f}) = \frac{1}{|X_{\text{test}}|} \sum_{x \in X_{\text{test}}} |y_{\text{test}}(x) - \tilde{f}(x)|^2.$$

Here $y_{\text{test}}(x)$ denotes the value of $x \in y_{\text{test}}$, that is given by our data set.

```

[21]: #model without automatic bandwidth detection
      #calculating mean squared error between our model and the test data
      mses = ANOVAapprox.get_mse( ads2)

      #choosing lambda with the least mse among all lambdas
      #only important, if multiple lambdas are given
      min_index = list(mses.keys())[np.argmin(list(mses.values()))]
      err_mse = mses[min_index]

      #model with automatic bandwidth detection
      #calculating mean squared error between our model and the test data
      mses_auto = ANOVAapprox.get_mse( ads_auto)

      #choosing lambda with the least mse among all lambdas
      #only important, if multiple lambdas are given
      min_index_auto = np.argmin(list(mses_auto.values()))
      err_mse_auto = mses_auto[min_index_auto]

```

Now, we compare both models by their errors. We will also have a look on the final attribute ranking and gsis.

At first, let's have a look on the errors given by each λ . We plot the error on the y -axis and the λ on the x -axis. If only one λ is given, then we will skip this part.

```
[22]: if len(lambdas) > 1:
    fig, (ax1, ax2) = plt.subplots(2,1)

    #plotting errors of model without automatic bandwidth detection
    ax1.scatter(lambdas, list(mses.values()), color = "orange")
    for a in range(len(lambdas)):
        ax1.plot([lambdas[a], lambdas[a]], [err_mse, mses[lambdas[a]]], color="orange")
    ax1.scatter(min_index, err_mse, color = "green", label = "$\\lambda_{\\min}$")

    ax1.set_xlabel("$\\lambda$")
    ax1.set_ylabel("mse")
    ax1.set_title("errors of model without automatic bandwidth detection")
    ax1.legend()

    #plotting errors of model with automatic bandwidth detection
    ax2.scatter(lambdas, list(mses_auto.values()), color = "orange")
    for a in range(len(lambdas)):
        ax2.plot([lambdas[a], lambdas[a]], [err_mse_auto, mses_auto[lambdas[a]]], color = "orange")
    ax2.scatter(min_index, err_mse, color = "green", label = "$\\lambda_{\\min}$")

    ax2.set_xlabel("$\\lambda$")
    ax2.set_ylabel("mse")
    ax2.set_title("errors of model with automatic bandwidth detection")
    ax2.legend()

    fig.subplots_adjust(hspace=0.4)
    plt.show()

    print("Without automatic bandwidth detection lambda = " + str(min_index) + "
    yields the best model with a mean squared error of " + str(err_mse) + ".")
    print("With automatic bandwidth detection lambda = " + str(min_index_auto) + "
    yields the best model with a mean squared error of " + str(err_mse_auto) + ".")
    print("=====
```

```

difference = err_mse - err_mse_auto

if difference < 0:
    print("The model without automatic bandwidth detection has an mse smaller_
↳than the mse with automatic bandwidth detection.")
    print("The difference between the two models is", difference, ", so the_
↳Model without automatic bandwidth detection has performed better.")
else:
    print("The model with automatic bandwidth detection has an mse smaller than_
↳the mse without automatic bandwidth detection.")
    print("The difference between the two models is", difference, ", so the_
↳Model with automatic bandwidth detection has performed better.")
print("We have an improvement of ", 100 - 100*err_mse_auto/err_mse, "%.")

```

Without automatic bandwidth detection $\lambda = 0.0$ yields the best model with a mean squared error of 0.012538001665914502.

With automatic bandwidth detection $\lambda = 0$ yields the best model with a mean squared error of 0.01651625351833854.

```

=====
=====

```

The model without automatic bandwidth detection has an mse smaller than the mse with automatic bandwidth detection.

The difference between the two models is -0.003978251852424038 , so the Model without automatic bandwidth detection has performed better.

We have an improvement of -31.72955275033351 %.

The following code lets you plot the attribute ranking and gsis only for the two λ that minimize the mean squared error for each model.

```

[23]: ri = ads2.get_AttributeRanking()
      gsis = ads2.get_GSI()
      ri_auto = ads_auto.get_AttributeRanking()
      gsis_auto = ads_auto.get_GSI()

      #creating raw plot
      fig, (ax1, ax2) = plt.subplots(2,1)

      #plotting attribute ranking
      #array for xaxis
      xaxis = np.arange(1, X.shape[0] + 1, 1)

      #plotting the attribute ranking

```

```

ax1.scatter(xaxis, ri[min_index], c = "orange", marker = "^", label =
    ↪"$\\lambda = $" + str(min_index))
ax1.scatter(xaxis + 0.2, ri_auto[min_index_auto], c = "green", marker = "^",
    ↪label = "$\\lambda = $" + str(min_index_auto))
for i in range(len(xaxis)):
    ax1.plot([i + 1, i + 1], [0, ri[min_index][i]], c = "orange")
    ax1.plot([i + 1.2, i + 1.2], [0, ri_auto[min_index_auto][i]], c = "green")

ax1.set_xticks(xaxis + (len(lambdas) - 1)*0.075)
ax1.set_xticklabels(xaxis - 1)
ax1.set_ylabel("$r(i)$")
ax1.set_title("attribute ranking")
ax1.set_xlim(0.75, len(ri[lambdas[0]]) + 1.5)
ax1.set_ylim(0, 1)
ax1.legend()
ax1.grid()

#plotting gsis
#array for xaxis
xaxis = np.arange(0, len(gsis[lambdas[0]]), 1)

#plotting the gsis
ax2.scatter(xaxis, gsis[min_index], c = "orange", marker = "^", label =
    ↪"$\\lambda = $" + str(min_index))
ax2.scatter(xaxis + 0.2, gsis_auto[min_index_auto], c = "green", marker = "^",
    ↪label = "$\\lambda = $" + str(min_index_auto))
for i in range(len(xaxis)):
    ax2.plot([i, i], [0, gsis[min_index][i]], c = "orange")
    ax2.plot([i + 0.2, i + 0.2], [0, gsis_auto[min_index_auto][i]], c = "green")

ax2.set_xticks(xaxis + (len(lambdas) - 1)*0.075)
ax2.set_xticklabels([tuple(x for x in tup) for tup in ads2.get_ActiveSet(np.
    ↪zeros((ds)), lam = lambdas[0])[1:]])
ax2.set_ylabel("$\\rho(\\mathbf{u}, S(X, I(U)) f)$")
ax2.set_title("global sensitivity indices")
ax2.set_xlim(-0.5, len(gsis[lambdas[0]]) + 1.5)
ax2.legend()
ax2.grid()

fig.set_figwidth(xaxis.shape[0]/1.2)
fig.set_figheight(10)
fig.subplots_adjust(hspace=0.4)
plt.show()

```